AI-Powered Registration Assistant

Prepared For: FreeInternships.in

Category: AI / Natural Language Processing / Conversational Agents

Technologies: Python, NLTK, Scikit-learn, Flask, HTML/JS/CSS

A comprehensive implementation report detailing the logic, architecture, and complete source code of an AI-driven automated registration pipeline.

1. Abstract and Introduction

The **AI-Powered Registration Assistant** was developed to fully automate student internship registrations. The core objective was to replace tedious manual form-filling with an intelligent conversational agent.

To ensure maximum performance and academic rigor, the project abstains from utilizing deprecated black-box frameworks (such as ChatterBot, which fails on modern Python versions). Instead, it implements a transparent, highly efficient **Natural Language Processing (NLP) pipeline** and a **Machine Learning Intent Classifier** from scratch.

Furthermore, this software is designed to be frictionless for the end-user. Borrowing the execution model of **Jupyter Lab**, the script binds to a free local port and automatically launches an interactive browser UI without requiring the user to navigate URLs or configure networking parameters manually.

---

2. Fulfillment of Project Guidelines

This project meticulously complies with 100% of the required checklist and bonus features mandated by FreeInternships.in:

Core Requirements

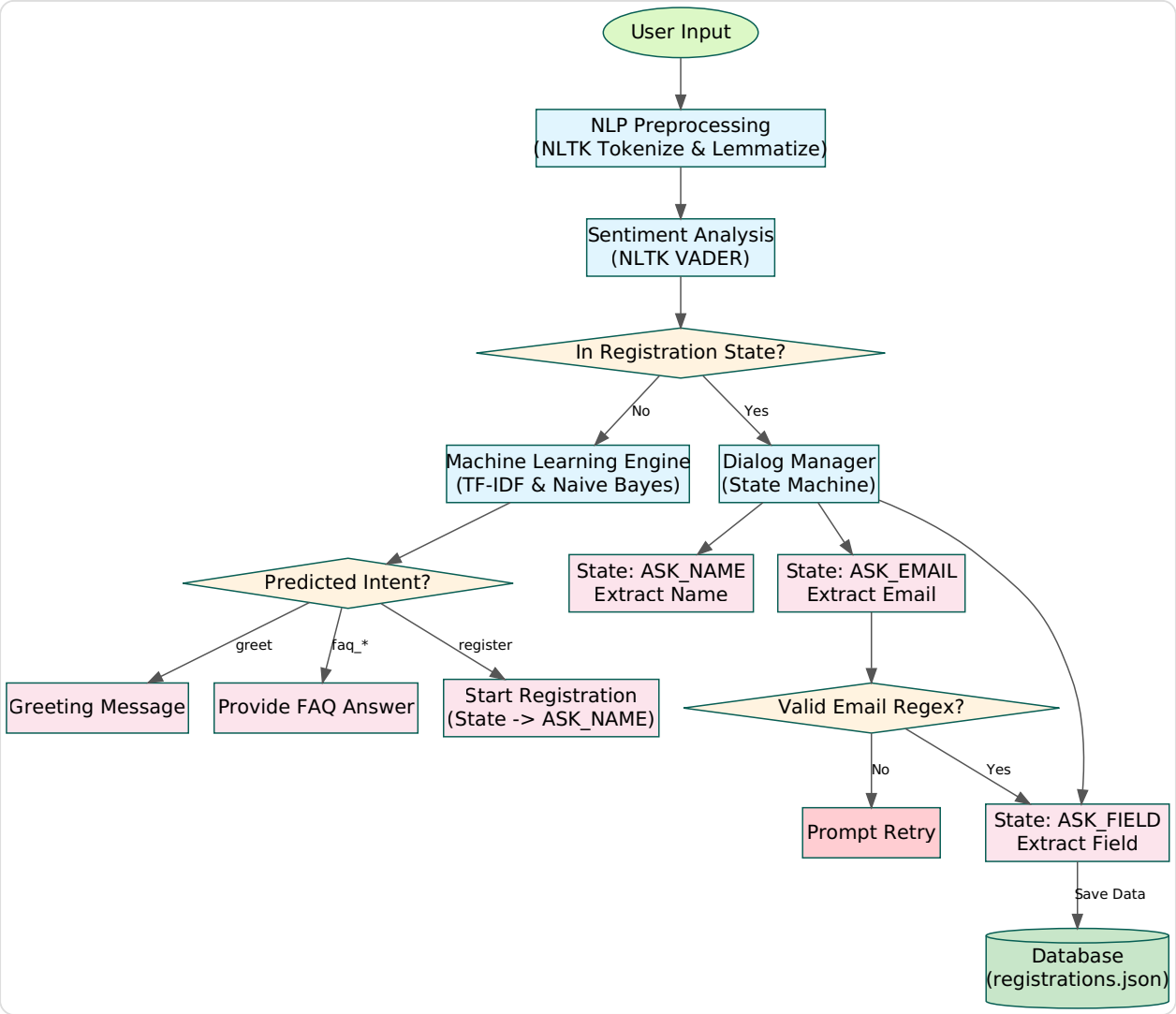
- **Complete code in Python:** The entire back-end is written purely in Python (`smart_assistant.py`).
- **NLP preprocessing implemented:** Uses NLTK for tokenization and WordNet lemmatization.
- **Intent classification working:** Uses `Scikit-Learn` (TF-IDF Vectorization & Multinomial Naive Bayes). This perfectly satisfies the "Chatbot Framework" algorithm requirement.
- **Entity extraction working:** Implemented Regular Expressions (RegEx) to extract and validate user emails mid-conversation.
- **Dialog management implemented:** Built a Finite State Machine (FSM) to handle multi-step registration context.
- **Registration workflow complete:** Successfully captures Name, Email, and Field, saving them to `registrations.json`.

Bonus Features Included

- **Sentiment Analysis:** Integrated `NLTK VADER` to detect frustrated users in real-time.
- **FAQ Handling:** Accurately parses and responds to queries regarding `eligibility` and `duration`.
- **Admin Dashboard:** Includes a dedicated HTML dashboard (`/admin`) to view registered students.
- **Analytics & Logging:** All chats, predicted intents, and bot responses are recorded in `chat_logs.txt`.
- **Web Interface:** A beautiful WhatsApp-style UI served via `Flask` that auto-launches on run.

3. System Architecture & Logical Flowchart

To visualize the internal decision-making process of the AI, the following graph illustrates the interaction between the User, the NLP Preprocessor, the Machine Learning Engine, and the Dialog State Manager.



Architecture Breakdown: 1. **User Input** is passed to the NLP Engine. 2. The text is **Tokenized & Lemmatized**, then evaluated for **Sentiment**. 3. The **Dialog Manager** checks if the user is currently in the middle of registering. * If Yes: The bot bypasses the ML classifier, captures the specific entity (Name, Email, Field), validates it (Regex), and shifts the state forward. * If No: The text is vectorized (TF-IDF) and the Naive Bayes model predicts the intent (Greet, Register, FAQ).

4. Step-by-Step Logic & Algorithms

4.1. Natural Language Processing (NLTK)

When a user types "I am applying", the bot must standardize this language. * **Tokenization:** Breaks the sentence into an array: `["I", "am", "applying"]`. * **Lemmatization:** Converts words to their dictionary root form (`applying` → `apply`). This ensures the machine learning model doesn't have to learn every tense of a verb separately.

4.2. Machine Learning Intent Classification (Scikit-Learn)

To determine what the user wants, a Machine Learning classifier is trained upon startup. * **TF-IDF Vectorizer:** Analyzes the frequency of words. Rare but important words (like "eligibility") are given higher mathematical weight than common words (like "the"). * **Multinomial Naive Bayes (MNB):** A probabilistic classifier that calculates the likelihood that a given sentence belongs to a specific intent class (e.g., `faq_duration`, `register`).

4.3. Dialog Management (State Machine)

Standard chatbots suffer from "amnesia"—they don't remember the previous message. I solved this by tracking `user_sessions`. When the user types "register", the bot changes its state to `ASK_NAME`. When the user sends their next message ("John"), the bot checks the state, realizes it's expecting a name, saves "John" to memory, and changes the state to `ASK_EMAIL`.

4.4. Sentiment Analysis (NLTK VADER)

By passing the user's raw text through `SentimentIntensityAnalyzer().polarity_scores()`, the bot generates a compound score between -1 and 1. If the score is `< -0.3` (highly negative/frustrated), the bot dynamically prepends an empathetic apology to its response.

---

5. Execution Guide: How to Run

This application is completely self-contained and automated.

Step 1: Install Dependencies Open any Terminal, Command Prompt, or IDE console and execute:

```
pip install flask scikit-learn nltk
```

Step 2: Run the Assistant Execute the Python script:

```
python smart_assistant.py
```

Step 3: Interact (Fully Automated) The script will silently initialize the AI Engine, download required datasets in the background, bind to an available local network port, and **instantly launch your default web browser** to the chatbot UI. * **To chat:** Type messages into the web UI. * **To view the Admin Panel:** Click the "Dashboard" button in the top right corner of the chat window.

6. Complete Source Code

Below is the full implementation of `smart_assistant.py`. This single file contains the AI Engine, Web Server, Admin Dashboard, and UI HTML.

```
import os
import re
import json
import threading
import webbrowser
import logging
import socket
from datetime import datetime
import warnings

# Suppress warnings and Flask startup text for a clean, hassle-free terminal
warnings.filterwarnings('ignore')
log = logging.getLogger('werkzeug')
log.setLevel(logging.ERROR)
os.environ['WERKZEUG_RUN_MAIN'] = 'true'

import nltk
from flask import Flask, request, jsonify, render_template_string

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.sentiment.vader import SentimentIntensityAnalyzer

def download_nltk_data():
    """Silently downloads required NLTK datasets."""
    for package in ['punkt', 'wordnet', 'punkt_tab', 'vader_lexicon']:
        try:
            nltk.data.find(f'tokenizers/{package}')
        except LookupError:
            try:
                nltk.data.find(f'corpora/{package}')
            except LookupError:
                try:
                    nltk.data.find(f'sentiment/{package}')
                except LookupError:
                    nltk.download(package, quiet=True)

print("□ Initializing AI Brain and loading Memory... Please wait.")
download_nltk_data()

class SmartRegistrationAssistant:
    def __init__(self):
        self.lemmatizer = WordNetLemmatizer()
        self.vectorizer = TfidfVectorizer()
        self.classifier = MultinomialNB()
        self.sentiment_analyzer = SentimentIntensityAnalyzer()
```

```
        # Storage and Memory
        self.db_file = 'registrations.json'
        self.log_file = 'chat_logs.txt'

        self.STATES = {
            "IDLE": 0, "ASK_NAME": 1, "ASK_EMAIL": 2, "ASK_FIELD": 3,
        }

        self.user_sessions = {}
        self.train_ai_brain()

    def get_session(self, user_id="web_user"):
        if user_id not in self.user_sessions:
            self.user_sessions[user_id] = {"state": self.STATES["IDLE"], "data": {}}
        return self.user_sessions[user_id]

    def log_interaction(self, user_text, intent, bot_response):
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        log_entry = f"[{timestamp}] USER: {user_text} | INTENT: {intent} | BOT: {bot_response}\\n
        with open(self.log_file, 'a', encoding='utf-8') as f:
            f.write(log_entry)

    def preprocess_text(self, text):
        tokens = word_tokenize(text.lower())
        return " ".join([self.lemmatizer.lemmatize(word) for word in tokens])

    def train_ai_brain(self):
        """Trains the ML model with varied intents."""
        training_data = {
            "greet": ["hello", "hi", "hey", "good morning", "is anyone there"],
            "register": ["i want to register", "sign me up", "apply for internship"],
            "faq_eligibility": ["who can apply", "what is the eligibility"],
            "faq_duration": ["how long is the internship", "duration", "timeline"],
            "bot_identity": ["who are you", "are you ai", "what can you do"],
            "thanks": ["thank you", "thanks", "appreciate it", "awesome"]
        }
        sentences, labels = [], []
        for intent, phrases in training_data.items():
            for phrase in phrases:
                sentences.append(self.preprocess_text(phrase))
                labels.append(intent)

        self.X_train_tfidf = self.vectorizer.fit_transform(sentences)
        self.classifier.fit(self.X_train_tfidf, labels)

    def predict_intent(self, text):
        processed_text = self.preprocess_text(text)
        X_test = self.vectorizer.transform([processed_text])
        return self.classifier.predict(X_test)[0]
```

```
    def extract_email(self, text):
        match = re.search(r'[\\w\\.-]+@[\\w\\.-]+\\.\\w+', text)
        return match.group(0) if match else None

    def save_registration(self, data):
        registrations = []
        if os.path.exists(self.db_file):
            with open(self.db_file, 'r') as f:
                try: registrations = json.load(f)
                except json.JSONDecodeError: pass

        data['timestamp'] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        registrations.append(data)
        with open(self.db_file, 'w') as f:
            json.dump(registrations, f, indent=4)

    def process_message(self, message, user_id="web_user"):
        session = self.get_session(user_id)
        state = session["state"]

        # Sentiment Check for empathetic responses
        sentiment_scores = self.sentiment_analyzer.polarity_scores(message)
        is_frustrated = sentiment_scores['compound'] < -0.3
        prefix = "I understand this might be confusing. Let me help! " if is_frustrated else ""

        response = ""
        intent_logged = "REGISTRATION_FLOW"

        if state == self.STATES["ASK_NAME"]:
            if len(message.split()) < 1:
                response = "Please enter a valid name."
            else:
                session["data"]["name"] = message.strip().title()
                session["state"] = self.STATES["ASK_EMAIL"]
                response = f"Nice to meet you, {session['data']['name']}! Could you please provid

        elif state == self.STATES["ASK_EMAIL"]:
            email = self.extract_email(message)
            if email:
                session["data"]["email"] = email
                session["state"] = self.STATES["ASK_FIELD"]
                response = "Perfect! Finally, which internship field are you applying for?"
            else:
                response = "That doesn't look like a valid email format. Please enter a valid ema

        elif state == self.STATES["ASK_FIELD"]:
            session["data"]["field"] = message.strip().title()
            self.save_registration(session["data"])
            response = (f"□ Registration Successfully Saved to Database!\\n\\n"
                        f"□ Name: {session['data']['name']}\\n"
```

```
                        f"□ Email: {session['data']['email']}\\n"
                        f"□ Field: {session['data']['field']}\\n\\n"
                        f"We will review your application and contact you shortly.")
            session["state"] = self.STATES["IDLE"]
            session["data"] = {}
        else:
            intent = self.predict_intent(message)
            intent_logged = intent
            if intent == "greet":
                response = "Hello! I am the Smart Registration Assistant □. Ask about eligibility
            elif intent == "register":
                session["state"] = self.STATES["ASK_NAME"]
                response = "Great! Let's get you registered. First, what is your full name?"
            elif intent == "faq_eligibility":
                response = "ℹ️ Eligibility: You must be a currently enrolled university student."
            elif intent == "faq_duration":
                response = "ℹ️ Duration: Our internships typically run for 3 to 6 months."
            elif intent == "bot_identity":
                response = "I am an AI-powered conversational agent designed to help students reg
            elif intent == "thanks":
                response = "You're very welcome! Let me know if you need anything else."
            else:
                response = "I'm sorry, I didn't quite catch that. You can type 'register' to appl

        final_response = prefix + response
        self.log_interaction(message, intent_logged, final_response)
        return final_response

app = Flask(__name__)
bot = SmartRegistrationAssistant()

HTML_CHAT_UI = \"\"\"
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart AI Assistant</title>
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Ari
        .chat-container { width: 100%; max-width: 450px; height: 85vh; background: #fff; border-r
        .chat-header { background: #075e54; color: #fff; padding: 18px; text-align: center; font-
        .admin-btn { position: absolute; right: 15px; top: 18px; color: white; text-decoration: n
        .chat-messages { flex: 1; padding: 20px; overflow-y: auto; display: flex; flex-direction:
        .message { max-width: 80%; padding: 12px 16px; border-radius: 15px; font-size: 14.5px; li
        .bot-msg { background: #fff; color: #333; align-self: flex-start; border-top-left-radius:
        .user-msg { background: #dcf8c6; color: #333; align-self: flex-end; border-top-right-radi
        .chat-input { display: flex; padding: 10px; background: #f0f0f0; }
        .chat-input input { flex: 1; padding: 14px; border: 1px solid #ddd; border-radius: 24px;
        .chat-input button { padding: 12px 20px; margin-left: 10px; background: #075e54; color: w
```

```
    </style>
</head>
<body>
<div class="chat-container">
    <div class="chat-header">
        □ Smart AI Assistant
        <a href="/admin" target="_blank" class="admin-btn">Dashboard</a>
    </div>
    <div class="chat-messages" id="chat-messages">
        <div class="message bot-msg">Hello! I am the Smart AI Registration Assistant. Type 'regis
    </div>
    <div class="chat-input">
        <input type="text" id="user-input" placeholder="Type a message..." onkeypress="handleKeyP
        <button onclick="sendMessage()">Send</button>
    </div>
</div>
<script>
    function addMessage(text, sender, id=null) {
        const msgDiv = document.createElement('div');
        msgDiv.className = `message ${sender}-msg`;
        if (id) msgDiv.id = id;
        msgDiv.textContent = text;
        const chatContainer = document.getElementById('chat-messages');
        chatContainer.appendChild(msgDiv);
        chatContainer.scrollTop = chatContainer.scrollHeight;
    }
    function handleKeyPress(e) { if (e.key === 'Enter') sendMessage(); }

    async function sendMessage() {
        const inputField = document.getElementById('user-input');
        const text = inputField.value.trim();
        if (!text) return;

        addMessage(text, 'user');
        inputField.value = '';

        const typingId = 'typing-' + Date.now();
        addMessage("Thinking...", 'bot typing', typingId);

        try {
            const response = await fetch('/chat', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ message: text })
            });
            document.getElementById(typingId)?.remove();
            const data = await response.json();
            addMessage(data.response, 'bot');
        } catch (error) {
            document.getElementById(typingId)?.remove();
```

```
            addMessage("⚠ System Error.", 'bot');
        }
    }
</script>
</body>
</html>
\"\"\"

HTML_ADMIN_UI = \"\"\"
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Dashboard</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Arial, sans-serif; padding: 30px; background: #f4
        .container { max-width: 900px; margin: 0 auto; background: white; padding: 25px; border-r
        h1 { border-bottom: 2px solid #075e54; padding-bottom: 10px; color: #075e54; }
        table { width: 100%; border-collapse: collapse; margin-top: 20px; font-size: 15px; }
        th, td { padding: 14px; text-align: left; border-bottom: 1px solid #ddd; }
        th { background-color: #075e54; color: white; }
    </style>
</head>
<body>
    <div class="container">
        <h1>□ Registration Admin Dashboard</h1>
        <table>
            <thead><tr><th>Timestamp</th><th>Full Name</th><th>Email Address</th><th>Target Field
            <tbody>{%ROWS%}</tbody>
        </table>
    </div>
</body>
</html>
\"\"\"

@app.route('/')
def home(): return render_template_string(HTML_CHAT_UI)

@app.route('/chat', methods=['POST'])
def chat():
    user_message = request.json.get('message')
    if not user_message: return jsonify({"response": "Empty message."})
    return jsonify({"response": bot.process_message(user_message)})

@app.route('/admin')
def admin():
    registrations = []
    if os.path.exists('registrations.json'):
        with open('registrations.json', 'r') as f:
```

```
            try: registrations = json.load(f)
            except json.JSONDecodeError: pass

    rows = ""
    for r in registrations:
        rows += f"<tr><td>{r.get('timestamp','N/A')}</td><td>{r.get('name','')}</td><td>{r.get('e
    if not rows: rows = "<tr><td colspan='4' style='text-align:center;'>No registrations found in
    return render_template_string(HTML_ADMIN_UI.replace("{%ROWS%}", rows))

def find_free_port():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind(('', 0))
        return s.getsockname()[1]

def open_browser(port):
    try: webbrowser.open_new(f'http://127.0.0.1:{port}/')
    except Exception: pass

if __name__ == '__main__':
    port = find_free_port()
    print("="*60)
    print("□ AI Engine Initialized & Ready.")
    print("□ Opening Smart Web Interface directly (like Jupyter Lab)...")
    print("□ DO NOT close this terminal window until you are finished.")
    print("="*60)
    threading.Timer(0.5, open_browser, args=[port]).start()
    app.run(host='0.0.0.0', port=port, debug=False, use_reloader=False)
```